

OBSERVACIONES DE LA PRÁCTICA

Estudiante 1 Cod 202610140
Estudiante 2 Cod 202614247
Estudiante 3 Cod XXXX

Ambientes de pruebas

	Máquina 1	Máquina 2	Máquina 3
Procesadores	12 th Gen Intel(R) Core (TM) i5-1235U (1.30 GHz)	Intel(R) Core(TM) Ultra 5 235U (2.00 GHz)	
Memoria RAM (GB)	16	16	
Sistema Operativo	Windows 11	Windows 11	

Tabla 1. Especificaciones de las máquinas para ejecutar las pruebas de rendimiento.

Máquina 1

Resultados para Queue con Array List

Porcentaje de la muestra	enqueue (Array List)	dequeue (Array List)	peek (Array List)
0.50%	0.074	0.051	0.003
5.00%	0.257	0.328	0.002
10.00%	0.383	0.373	0.001
20.00%	1.231	9.666	0.002
30.00%	1.813	20.632	0.002
50.00%	2.633	63.511	0.001
80.00%	3.708	109.787	0.001
100.00%	5.679	119.023	0.002

Resultados para Stack con Array List

Porcentaje de la muestra	push (Array List)	pop (Array List)	top(Linked List)
0.50%	0.075	0.044	0.017
5.00%	0.354	0.198	0.065
10.00%	0.474	0.376	0.127
20.00%	1.509	1.522	0.377
30.00%	3.795	2.877	0.516
50.00%	2.21	1.575	0.468
80.00%	5.604	3.132	1.428

100.00%	9.776	5.034	3.49
----------------	--------------	--------------	-------------

Resultados para Queue con Linked List

Porcentaje de la muestra	enqueue (Linked List)	dequeue (Linked List)	peek Linked List)
0.50%	0.092	0.139	0.011
5.00%	0.495	0.388	0.005
10.00%	0.84	0.785	0.003
20.00%	2.416	1.789	0.002
30.00%	4.677	4.627	0.004
50.00%	5.029	3.843	0.003
80.00%	6.544	5.95	0.005
100.00%	5.809	5.426	0.003

Resultados para Stack con Linked List

Porcentaje de la muestra	push (Linked List)	pop (Linked List)	top(Linked List)
0.50%	0.049	0.045	0.025
5.00%	0.357	0.492	0.174
10.00%	0.745	0.808	0.353
20.00%	1.92	1.835	0.784
30.00%	2.82	2.297	1.291
50.00%	3.678	1.474	3.633
80.00%	11.536	6.723	2.868
100.00%	6.871	7.55	2.658

Máquina 2

Resultados para Queue con Array List

Porcentaje de la muestra	enqueue (Array List)	dequeue (Array List)	peek (Array List)
0.50%	0.090	0.142	0.006
5.00%	0.629	0.486	0.008
10.00%	0.794	1.072	0.003
20.00%	1.620	2.445	0.003
30.00%	2.214	3.274	0.003
50.00%	3.819	9.408	0.003
80.00%	6.765	30.176	0.004

100.00%	5.213	64.265	0.003
----------------	-------	--------	-------

Resultados para Stack con Array List

Porcentaje de la muestra	push (Array List)	pop (Array List)	top(Array List)
0.50%	0.057	0.049	0.029
5.00%	0.536	0.468	0.238
10.00%	1.260	1.026	0.481
20.00%	3.455	2.449	0.955
30.00%	5.262	4.353	1.183
50.00%	8.220	7.514	0.897
80.00%	36.021	101.313	2.929
100.00%	29.505	158.614	1.694

Resultados para Queue con Linked List

Porcentaje de la muestra	enqueue (Linked List)	dequeue (Linked List)	peek Linked List)
0.50%	0.063	0.027	0.003
5.00%	0.941	0.512	0.013
10.00%	1.113	0.617	0.003
20.00%	2.493	2.093	0.005
30.00%	3.809	3.362	0.004
50.00%	5.284	4.300	0.002
80.00%	10.545	9.384	0.004
100.00%	24.628	13.743	0.004

Resultados para Stack con Linked List

Porcentaje de la muestra	push (Linked List)	pop (Linked List)	top(Linked List)
0.50%	0.040	0.026	0.016
5.00%	0.546	0.508	0.244
10.00%	0.627	0.551	0.263
20.00%	2.105	2.079	0.979
30.00%	3.609	3.207	1.485
50.00%	3.855	3.430	2.943
80.00%	9.933	9.085	4.482
100.00%	12.564	7.026	3.946

Máquina 3

Resultados para Queue con Array List

Porcentaje de la muestra	enqueue (Array List)	dequeue (Array List)	peek (Array List)
0.50%			
5.00%			
10.00%			
20.00%			
30.00%			
50.00%			
80.00%			
100.00%			

Resultados para Stack con Array List

Porcentaje de la muestra	push (Array List)	pop (Array List)	top(Array List)
0.50%			
5.00%			
10.00%			
20.00%			
30.00%			
50.00%			
80.00%			
100.00%			

Resultados para Queue con Linked List

Porcentaje de la muestra	enqueue (Linked List)	dequeue (Linked List)	peek Linked List)
0.50%			
5.00%			
10.00%			
20.00%			
30.00%			
50.00%			
80.00%			
100.00%			

Resultados para Stack con Linked List

Porcentaje de la muestra	push (Linked List)	pop (Linked List)	top(Linked List)
0.50%			
5.00%			
10.00%			
20.00%			
30.00%			
50.00%			
80.00%			
100.00%			

Preguntas de análisis

1. ¿Se observan diferencias significativas entre las implementaciones con ArrayList y LinkedList para las funciones de Queue y Stack? ¿Cuál es más eficiente en cada operación? ¿Por qué una implementación es más rápida en ciertos casos?
2. ¿Cuándo es preferible usar ArrayList o LinkedList? Si insertamos y eliminamos con frecuencia, ¿qué estructura conviene más? Si accedemos aleatoriamente a elementos, ¿cuál es más eficiente?
3. Durante la ejecución de las pruebas ¿Se presentan anomalías en los tiempos de ejecución que no se explican con la teoría?
4. Complete la siguiente tabla de acuerdo con qué operación es más eficiente en cada implementación (marque con una x la que es más eficiente). Adicionalmente, explique si este comportamiento es acorde con lo enunciado en la teoría. Justifique las respuestas.

1.

Sí se observan diferencias significativas, y estas aumentan con el tamaño de la muestra. En Array List, insertar o eliminar en el inicio de la estructura tiene una complejidad de $O(n)$, debido a que requiere desplazar todos los elementos restantes; en Linked List la misma operación es de $O(1)$, ya que solo se actualiza un puntero. Por esto, los tiempos de Array List crecen considerablemente a medida que aumenta la muestra, mientras que los de Linked List se mantienen medianamente constantes. La operación de consulta (peek/top) es $O(1)$ en ambas implementaciones, por lo que no presenta diferencias significativas. En general, Linked List resulta más eficiente para los insert y delete en los extremos, mientras que Array List solo iguala esa eficiencia cuando la operación se realiza al final del arreglo.

2.

Cuando los insert y delete son frecuentes, especialmente al inicio o en posiciones intermedias, resulta más conveniente una Linked List, dado que su complejidad es $O(1)$ al no necesitar un desplazamiento de elementos. Cuando se requiere acceso aleatorio a elementos por posición, resulta más eficiente una Array List, ya que permite acceso directo en $O(1)$, mientras que una Linked List requiere recorrer la estructura secuencialmente en $O(n)$.

3.

Sí, se ven algunas anomalías en los tiempos. En algunas pruebas el tiempo baja aunque la muestra sea más grande, lo cual no sería lo esperado según la teoría. Esto puede pasar porque mientras se ejecuta el programa también influyen otras cosas del computador, como procesos en segundo plano, uso de memoria o pequeñas variaciones al medir el tiempo. Por eso los resultados no siempre salen de forma totalmente uniforme, aunque en general sí se nota la tendencia esperada.

		Array List	Linked List	Justificación
QUEUE	Enqueue()	X		En nuestras pruebas Array List presentó menores tiempos en la mayoría de las muestras. Esto es razonable, ya que agregar al final es una operación eficiente y no requiere mover los demás elementos.
	Dequeue()		X	Linked List fue más eficiente, sobre todo con muestras grandes. Esto coincide con la teoría porque para eliminar el primero solo se cambia la referencia al siguiente nodo, mientras que en Array List se deben desplazar los elementos.
	Peek()	X		Array List tuvo tiempos ligeramente menores en la mayoría de las pruebas. Según la teoría, en ambas estructuras esta operación es muy rápida porque solo se consulta el primer elemento, así que la diferencia observada es pequeña.
STACK	Push()		X	Linked List es más conveniente porque agregar un elemento al inicio solo requiere modificar referencias. En Array List es necesario desplazar los elementos existentes.
	Pop()		X	Linked List resulta más eficiente porque puede eliminar directamente el primer nodo. En Array List se deben mover los elementos restantes, por lo que el costo aumenta con muestras grandes.
	Top()	X		En las pruebas Array List tuvo mejores tiempos en general. Sin embargo, según la teoría, en las dos implementaciones la operación es $O(1)$, porque solo se consulta el primer elemento sin eliminarlo. Por eso la diferencia entre ambas no debería ser muy grande.

